

Matematica

TPI 1

Nombres: Lautaro Maldonado, Tiziano Joaquin Hidalgo, Quiroga Tomas, Alejo Metillo, Joaquin Ferreyra y Genaro Oliveira

Consigna 1 – Parte A: Análisis con Conjuntos

Datos

Sea:

$$A = \{101, 102, 103, 104, 105, 106\}$$

Usuarios que acceden mediante la API.

$$B = \{104, 105, 106, 107, 108\}$$

Usuarios que acceden mediante la Web.

$$C = \{102, 105, 109\}$$

Usuarios que generaron errores en el sistema.

1.a) Usuarios que utilizan ambas plataformas

Para encontrar los usuarios que utilizan tanto la API como la Web se calcula la intersección de los conjuntos A y B.

$$A \cap B$$

Buscamos los elementos que aparecen en ambos conjuntos:

$$A = \{101, 102, 103, 104, 105, 106\}$$

$$B = \{104, 105, 106, 107, 108\}$$

Los elementos comunes son:

$$A \cap B = \{104, 105, 106\}$$

Interpretación

Los usuarios 104, 105 y 106 acceden a la plataforma tanto mediante la API como mediante la Web.

1.b) Usuarios que utilizan al menos una plataforma

Para obtener todos los usuarios que utilizan alguna de las plataformas se calcula la unión entre A y B.

AUB

Se incluyen todos los elementos sin repetir:

$$A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$$

Interpretación

Estos usuarios utilizan la API, la Web o ambas plataformas.

1.c) Usuarios que utilizan la plataforma pero no presentan errores

Primero calculamos la unión:

$$A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$$

Luego eliminamos los usuarios que pertenecen al conjunto C.

$$(A \cup B) - C$$

$$= \{101, 102, 103, 104, 105, 106, 107, 108\} - \{102, 105, 109\}$$

Eliminando 102 y 105 obtenemos:

$$(A \cup B) - C = \{101, 103, 104, 106, 107, 108\}$$

Interpretación

Los usuarios 101, 103, 104, 106, 107 y 108 utilizan alguna plataforma y no registran errores en el sistema.

1.d) Usuarios que utilizan exclusivamente una sola plataforma

Debemos calcular:

$$(A - B) \cup (B - A)$$

Paso 1: Calcular $A - B$

Son los usuarios que están en A pero no en B.

$$A - B = \{101, 102, 103\}$$

Paso 2: Calcular $B - A$

Son los usuarios que están en B pero no en A.

$$B - A = \{107, 108\}$$

Paso 3: Unir ambos resultados

$$(A-B) \cup (B-A)$$

$$= \{101, 102, 103\} \cup \{107, 108\}$$

$$= \{101, 102, 103, 107, 108\}$$

Interpretación

Estos usuarios utilizan únicamente una plataforma. No acceden simultáneamente por API y Web.

2. Expresar resultados mediante comprensión de conjuntos

Usuarios que utilizan ambas plataformas

$$A \cap B = \{x | x \in A \wedge x \in B\}$$

Interpretación: conjunto formado por todos los usuarios que pertenecen simultáneamente a A y B.

Usuarios que utilizan alguna plataforma y no presentan errores

$$(A \cup B) - C = \{x | x \in (A \cup B) \wedge x \in$$

/

$$C\}$$

Interpretación: conjunto formado por los usuarios que utilizan alguna plataforma y no registran errores.

3. Usuarios que aparecen en C pero no en AUB

Calculamos:

$$C - (A \cup B)$$

Sabemos que:

$$A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$$

Entonces:

$$\{102, 105, 109\} - \{101, 102, 103, 104, 105, 106, 107, 108\}$$

$$= \{109\}$$

Interpretación

El usuario 109 aparece registrado como generador de errores, pero no figura entre los usuarios que acceden mediante la API ni mediante la Web.

Esto puede indicar:

Un error en los registros del sistema.

Un problema de sincronización de datos.

Actividad proveniente de una fuente no registrada.

Un fallo en el mecanismo de monitoreo.

Como equipo de desarrollo sería necesario investigar este caso para verificar la consistencia de la información almacenada

Consigna 1 – Parte B: Lógica Proposicional

Definición de las proposiciones

Para modelar matemáticamente el problema se definen las siguientes proposiciones:

p: el usuario pertenece al conjunto A (usuarios que acceden mediante API).

q: el usuario pertenece al conjunto B (usuarios que acceden mediante la web).

r: el usuario pertenece al conjunto C (usuarios que han generado errores).

Se considera que un usuario es crítico cuando utiliza al menos una de las plataformas y además ha generado errores.

La expresión lógica correspondiente es:

$$(p \vee q) \wedge r$$

Justificación de la expresión

La expresión $p \vee q$ representa que el usuario pertenece a A o a B, es decir, utiliza al menos una de las plataformas.

La expresión anterior se combina con r mediante la conjunción ($\wedge$), ya que además debe cumplirse que el usuario haya generado errores.

Por lo tanto, un usuario será crítico únicamente cuando se cumplan simultáneamente ambas condiciones.

Tabla de verdad

p	q	r	$p \vee q$	$(p \vee q) \wedge r$
V	V	V	V	V
V	V	F	V	F
V	F	V	V	V

V	F	F	V	F
F	V	V	V	V
F	V	F	V	F
F	F	V	F	F
F	F	F	F	F

Interpretación de la tabla de verdad

A partir de la tabla se observa que la expresión lógica toma valor verdadero solamente cuando el usuario pertenece al menos a uno de los conjuntos A o B y, además, pertenece al conjunto C.

En todos los demás casos el resultado es falso, por lo que esos usuarios no son considerados críticos.

Implementación en Python

Para representar la expresión lógica se implementó la siguiente función:

```
def usuario_critico(p, q, r):
    if (p or q) and r:
        return True
    else:
        return False
```

Explicación de la lógica utilizada

La función recibe tres variables booleanas:

p = True si el usuario pertenece a A.

q = True si el usuario pertenece a B.

r = True si el usuario pertenece a C.

Si se cumple la condición

$(p \text{ or } q) \text{ and } r$

la función devuelve True, indicando que el usuario es crítico. En caso contrario devuelve False.

De esta manera, la expresión matemática se traduce directamente a lenguaje Python.

Clasificación de los usuarios

Se consideran los conjuntos:

$A = [101, 102, 103, 104, 105, 106]$

$B = [104, 105, 106, 107, 108]$

$C = [102, 105, 109]$

Se recorrieron todos los usuarios y se verificó si pertenecían o no a cada conjunto.

Resultado obtenido

Usuarios críticos:

102

105

Usuarios no críticos:

101

103

104

106

107

108

109

Interpretación de los resultados

Los usuarios 102 y 105 son considerados críticos porque cumplen la expresión lógica propuesta. Ambos utilizan alguna de las plataformas y además registran errores.

Por esta razón, estos usuarios requieren un seguimiento especial para evitar posibles fallos en el sistema.

Consigna 1 – Parte C: Interpretación

¿Qué tipo de usuario representa mayor riesgo?

Los usuarios críticos representan el mayor riesgo para el sistema.

Esto se debe a que cumplen la condición:

$$(p \vee q) \wedge r$$

Es decir, son usuarios activos dentro del sistema y al mismo tiempo generan errores.

En el dataset analizado, los usuarios críticos son:

102

105

Estos usuarios pueden afectar el funcionamiento del sistema y deben ser supervisados.

¿Qué significa que un usuario esté en C pero no en AUB?

Primero se calcula:

$$A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$$

y

$$C = \{102, 105, 109\}$$

Ahora se realiza la diferencia:

$$C - (A \cup B)$$

Comparando ambos conjuntos se obtiene:

$$C - (A \cup B) = \{109\}$$

Interpretación

El usuario 109 ha generado errores, pero no aparece registrado como usuario de API ni como usuario web.

Esto podría deberse a:

errores en la carga de datos;

inconsistencias en los registros;

problemas en el monitoreo;

accesos no identificados.

Por ello, este caso debe investigarse.

¿Qué decisión tomaríamos como equipo programador?

A partir de los resultados obtenidos, se propone:

1. Supervisar a los usuarios críticos

Los usuarios 102 y 105 presentan actividad y además generan errores, por lo que requieren un seguimiento especial.

2. Investigar al usuario 109

El hecho de que pertenezca a C pero no a AUB indica una posible inconsistencia en los registros.

3. Implementar controles adicionales

Se recomienda incorporar mecanismos de validación y monitoreo para detectar este tipo de situaciones y mejorar la seguridad y confiabilidad del sistema.

Consigna 2 – Parte A

1)

Para $A(x) = 40x + 200$:

- Pendiente (m): 40
- Ordenada al origen (b): 200 (en el gráfico va a ser el punto $(0, 200)$).

Para $B(x) = 70x + 50$:

- Pendiente (m): 70
- Ordenada al origen (b): 50 (en el gráfico va a ser el punto $(0, 50)$).

2) Dos rectas son paralelas si y solo si tienen la misma pendiente, como $40 \neq 70$ entonces podemos decir que no son paralelas. Al tener pendientes distintas, se van a cruzar en algún punto.

3) Calcular el punto de intersección entre A y B. Aplique el método de igualación, planteando que la función $A(x) = B(x)$: $40x + 200 = 70x + 50$

Paso los términos con x de un lado y los números solos del otro:

$$200 - 50 = 70x - 40x$$

Resuelvo cada lado:

$$150 = 30x$$

El punto de interseccion se va dar a las 5 horas, para ver cual es el costo asociado a ese tiempo remplazamos la x por 5 en las dos funciones:

$$x = \frac{150}{30}$$

$$x = 5 \text{ horas}$$

4) Para la funcion C calcular vertice:

Calculo de la coordenada x (las horas):

$$x_v = \frac{-80}{2 \cdot (-2)}$$

$$x_v = \frac{-80}{-4}$$

$$\mathbf{x_v = 20}$$

Calculo de la coordenada y (el costo):

$$y_v = -2 \cdot (20)^2 + 80 \cdot (20) + 100$$

$$y_v = -800 + 1600 + 100$$

$$y_v = 900$$

Resultado final: Uniendo ambas coordenadas el vertice de la parabola es el punto V = (20,900)

Para la funcion C calcular raiz. Utilizamos la formula resolvente de Bhaskara:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$x = \frac{-80 \pm \sqrt{80^2 - 4 \cdot (-2) \cdot 100}}{2 \cdot (-2)}$$

$$x = \frac{-80 \pm \sqrt{6400 + 800}}{-4}$$

$$x = \frac{-80 \pm \sqrt{7200}}{-4}$$

Utilizando el signo mas sacamos la primer raiz que nos da un numero negativo:

$$x_1 = \frac{-80 + 84.85}{-4} = \frac{4.85}{-4} \approx -1.21 \text{ horas}$$

Utilizando el signo menos sacamos la segunda raiz:

$$x_2 = \frac{-80 - 84.85}{-4} = \frac{-164.85}{-4} \approx 41.21 \text{ horas}$$

Parte B – Implementacion a python

La 5, 6, 7 y 8 estan en el codigo

```
    plan_a(x):  
    return 40 * x + 200  
  
def plan_b(x):  
    return 70 * x + 50  
  
def plan_c(x):  
    return -2 * (x * x) + 80 * x + 100  
dominio_x = []  
vector_a = []  
vector_b = []  
vector_c = []  
  
for x in range(51):  
    dominio_x.append(x)  
    vector_a.append(plan_a(x))  
    vector_b.append(plan_b(x))  
    vector_c.append(plan_c(x))  
  
print(" PUNTO 6: Estructura de datos para armar el gráfico ")  
print("Se muestran los puntos de 5 en 5 horas para resumir la tabla:")  
print("-" * 75)
```

```
print("Hora (X) | Puntos Plan A (X, Y) | Puntos Plan B (X, Y) | Puntos Plan C (X, Y)")
```

```
print("-" * 75)
```

```
for i in range(51):
```

```
    if i % 5 == 0:
```

```
        print(f"    {dominio_x[i]:2} |    ({dominio_x[i]:2}, {vector_a[i]:4}) |  
    ({dominio_x[i]:2}, {vector_b[i]:4}) |    ({dominio_x[i]:2}, {vector_c[i]:4})")
```

```
valores_prueba = [0, 5, 10, 15, 20, 25, 30, 40, 50]
```

```
for x in valores_prueba:
```

```
    print("Horas:", x, "-> A:", plan_a(x), "| B:", plan_b(x), "| C:", plan_c(x))
```

```
def obtener_mejor_plan(x):
```

```
    costo_a = plan_a(x)
```

```
    costo_b = plan_b(x)
```

```
    costo_c = plan_c(x)
```

```
    minimo = costo_a
```

```
    nombre = "Plan A"
```

```
    if costo_b < minimo:
```

```
        minimo = costo_b
```

```
        nombre = "Plan B"
```

```
    if costo_c < minimo:
```

```
        minimo = costo_c
```

```
        nombre = "Plan C"
```

```
    return nombre
```

```
contador_a = 0
```

```
contador_b = 0
```

```
contador_c = 0
```

```

for x in range(51):
    mejor = obtener_mejor_plan(x)

    if mejor == "Plan A":
        contador_a = contador_a + 1
    elif mejor == "Plan B":
        contador_b = contador_b + 1
    elif mejor == "Plan C":
        contador_c = contador_c + 1
horas_negativas = []

for x in range(51):
    if plan_c(x) < 0:
        horas_negativas.append(x)

print("El Plan C da costos negativos a partir de las horas:", horas_negativas)

```

Parte C – Analisis

9) Proyectos cortos (de 0 a 5 horas): Conviene el Plan B. Su costo fijo inicial es de solo \$50, el más bajo de todos. Proyectos medianos (de 5 a 35 horas): Conviene el Plan A. En este tramo, el Plan B se encarece mucho por su alta pendiente (\$70\$), y el Plan C sube hasta tocar un techo de \$900\$ en las 20 horas, por lo que la recta del Plan A se mantiene como la opción más económica. Proyectos largos (de 35 a 50 horas): Conviene el Plan C. A partir de las 35 horas, el costo del Plan C cae de forma picada, cruzando por debajo del Plan A y volviéndose la opción más barata para un alto volumen de horas.

10) Respuestas:

A) Valores donde el costo de c es negativo: $x_1 = -1.21$ (Se descarta, porque no existen horas de desarrollo negativas).

$x^2 = 41.21$ Como la parábola tiene las ramas hacia abajo (porque el término $-2x^2$ es negativo), la curva cruza el eje X hacia abajo en este punto.

B) En un contexto económico y comercial de desarrollo de software, este resultado representa un problema grave por dos razones:

Inviabilidad comercial (Contradicción económica): Que una función de costo sea menor que cero significa que el precio final se vuelve negativo. En el mundo real, esto implicaría que la empresa desarrolladora tendría que pagarle dinero al cliente por el simple hecho de contratar más horas de servicio.

Falla en el modelo matemático: Esto demuestra que la función cuadrática del Plan C tiene una limitación técnica importante. Las parábolas invertidas sirven para modelar costos con descuentos por volumen en tramos específicos, pero si se las deja libres sin una restricción o un "tope" en el dominio, terminan decayendo hasta el infinito.

Actividad 3

Parte A

- 1) Cada fila de M representa la función del sistema, y las columnas representan los servidores, y un valor (por ejemplo 120) representa el tiempo que se tarda en ejecutar dicha función en ese servidor
- 2) Es posible calcular el producto $M \cdot C$ porque M al tener 3 filas y 3 columnas, tiene la misma dimensión que C, pudiendo multiplicar fila*columna y después sumando todos los resultados de esas multiplicaciones para el resultado final

Parte B

Código completo (muestra el tiempo promedio de ejecución por función y por servidor, además del método para calcular la matriz compuesta de M):

#Promedio de ejecución por función:

```
m = [  
    [120, 150, 110],  
    [200, 180, 220],  
    [90, 110, 95]  
]
```

```
for i in range(3):
```

```
    sumafilas = 0
```

```

for j in range(3):
    sumafila = sumafila + m[i][j]

promediofila = sumafila / 3

print(f"Promedio de la fila {i + 1}: {promediofila}")

#Promedio de ejecucion por servidor:

for i in range(3):
    sumacolumna = 0
    for j in range(3):
        sumacolumna = sumacolumna + m[j][i]

    promediocolumna = sumacolumna / 3

    print(f"Promedio de la columna {i + 1}: {promediocolumna}")

#Metodo para sacar la matriz compuesta

mtranspuesta = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]

for i in range(3):
    for j in range(3):
        mtranspuesta[j][i] = m[i][j]

print("Matriz Transpuesta Correcta:")

for fila in mtranspuesta:
    print(fila)

```

En este contexto la matriz compuesta representaria cada fila un servidor distinto y cada columna una funcion distinta

Parte C

5) Producto calculado:

Parte D

6) analisis: Cada valor de la matriz T representa la combinacion de los tiempos de ejecucion de la matriz M en todos los servidores con las cantidades de ejecuciones que se hicieron de todas las funciones del servidor C

No tiene sentido practico

7) no tiene sentido porque en este problema las columnas de M son servidores y las filas de C son funciones. Al multiplcar todo esto estamos cruzando los datos de forma que no tenga sentido alguno.

La forma correcta de hacerlo seria multiplicar directamente los los datos de M con C y poner el resultado en T, por ejemplo, multiplicar 120×30 , que serian los datos ubicados en la fila 1 y columna 1 y a ese resultado ponerlo en T

Parte E

8) la matriz es simetrica, esto debido a que tiene la misma cantidad tanto de columnas como de filas

9) La matriz es invertible porque su determinante es distinto de cero, lo que significa que sus filas (y columnas) son linealmente independientes y la matriz es de rango completo (rango 3).

10) Si la matriz M no fuera invertible, significaría matemáticamente que sus filas o columnas son **linealmente dependientes**. Que sea invertible nos indica que cada servidor y funcion se comportan de manera independiente y unica de cada uno.

Parte F

11) la funcion que presenta mayor costo computacional promedio es la funcion 2, y el servidor mas eficiente seria el servidor 1, basandonos en los resultados de los calculos del codigo que hicimos anteriormente

12) una recomendacion que haríamos seria rebalancear todas las funciones en los 3 servidores para tratar de aligerar la carga de todos